

Vi / Vim — Command Line Text Editor

Section 6 — Detailed Command Reference | nezuko homelab

Overview

Vi is a **modal text editor** available on every Linux distribution, Unix system, macOS, minimal Docker containers, embedded systems, and routers. It is the one editor guaranteed to exist everywhere. Vim (Vi IMproved) is a superset of vi with extra features. On Ubuntu, **vi is actually vim in disguise** via symlinks — but in production containers you will get true minimal vi (often via BusyBox) with no extras.

Vi vs Vim at a glance:

Aspect	vi	vim
Full name	Visual editor	Vi IMproved
Availability	Everywhere — guaranteed	Most systems
Undo	Single level only	Multiple levels
Syntax highlighting	No	Yes
INSERT mode indicator	Not always	Yes (-- INSERT --)

■ Always write scripts and configs using only commands that work in basic vi. Relying on vim-only features will break things in containers and embedded systems.

The Modal Concept — Most Important Thing to Understand

Vi is a **modal editor** — the same key does completely different things depending on which mode you are currently in. There are two modes you need to know:

Mode	Prompt indicator	How to enter	What keys do
Normal	No indicator (default)	Press Esc from anywhere	Trigger commands — navigate, delete, undo, search
Insert	-- INSERT -- (vim only)	Press i, a, o, O, or A	Type text into the file

■ When in doubt — press Esc. You can never go wrong. Esc always returns you to Normal mode safely from anywhere.

Command: vi / busybox vi

vi

Open or create a file in the vi text editor

What it does:

Opens a file for editing in vi. If the file does not exist, vi creates it when you save. On Ubuntu, **vi** is **symlinked to vim.basic** — so you get vim features automatically. In Docker containers and embedded systems you will get true minimal vi via BusyBox.

Common options:

Option	What it does
vi filename	Open or create a file
vi +N filename	Open file and jump to line number N
vi +/pattern filename	Open file and jump to first match of pattern
vi -R filename	Open file in read-only mode (cannot save changes)
busybox vi filename	Open with BusyBox's minimal vi — simulates containers

Examples on nezuko:

```
vi test.txt # Open or create test.txt
vi /etc/hosts # Open hosts file (needs sudo to save)
sudo vi /etc/netplan/00-installer-config.yaml # Edit nezuko's network config
vi +10 test.txt # Open and jump to line 10
vi -R /etc/passwd # View passwd safely – cannot save
busybox vi test.txt # Open with minimal vi for container practice
```

Every option explained:

Option	Full explanation
vi filename	Standard way to open a file. Creates the file on first :w or :wq if it did not exist.
vi +N filename	Positions cursor at line N on open. Useful when you know exactly which line has the problem.
vi +/pattern filename	Searches for pattern on open and positions cursor at first match. Useful for jumping to a known config.
vi -R filename	Read-only mode. Prevents accidental edits to important files. :w and :wq will be refused.
busybox vi filename	BusyBox stripped-down vi. No INSERT indicator, no syntax highlighting, single undo. Exactly what you need.

Insert Mode Entry Commands

i a o O A

Enter Insert mode from Normal mode — different entry points

What they do:

All of these switch from Normal mode to Insert mode, but each positions the cursor differently before you start typing. Choosing the right one saves you moving the cursor manually.

All insert entry options:

Command	Where cursor goes	Best used when...
i	Before current cursor position	You want to insert text at the cursor
a	After current cursor position (append)	You want to add text after the current character
o	New blank line BELOW current line	You want to add a new line below and start typing
O	New blank line ABOVE current line	You want to add a new line above and start typing
A	End of current line	You want to add text to the end of a line

Examples on nezuko:

```
# Editing /home/scottp/config.txt:
i # Insert before cursor – add text mid-line
a # Append after cursor – add a character after current position
o # New line below – most common for adding new config entries
O # New line above – useful to insert before current line
A # Append to end of line without navigating there first
# Always press Esc when done typing to return to Normal mode
```

Command: Esc

Esc

Exit Insert mode and return to Normal mode

What it does:

The single most important key in vi. Pressing Esc from anywhere — Insert mode, mid-command, anywhere at all — returns you safely to Normal mode. If you are ever unsure what mode you are in, press Esc and you will be back in Normal mode.

Key behaviour:

Situation	What Esc does
In Insert mode	Returns to Normal mode — stops typing
Already in Normal mode	Does nothing — completely safe to press
Mid-command (e.g. half typed :wq)	Cancels the command and returns to Normal mode
In BusyBox vi (no INSERT indicator)	Same behaviour — essential since you cannot see the mode

Save and Quit Commands (: commands)

:w :wq :q :q! ZZ

Save and/or quit the editor — typed in Normal mode

What they do:

Colon commands are typed in Normal mode. When you press : you will see the colon appear at the bottom of the screen — then type the rest of the command and press Enter. These control whether your changes are saved and whether vi closes.

All save and quit options:

Command	What it does	When to use it
:w	Save (write) the file — stay in vi	Mid-edit saves — save progress without quitting
:wq	Save and quit	Normal way to finish editing
:q	Quit without saving	Only works if no changes have been made
:q!	Quit and discard ALL changes	Made a mess and want to abandon everything
ZZ	Save and quit (no colon needed)	Shortcut — same as :wq
:w filename	Save a copy to a new filename	Save current content under a different name

Examples on nezuko:

```
# Standard workflow:
vi config.txt # Open file
i # Insert mode
timeout=30 # Type your change
Esc # Back to Normal mode
:wq # Save and quit

# Emergency exit – abandon all changes:
Esc # Make sure in Normal mode
:q! # Quit without saving – nothing is changed

# Save mid-edit without quitting:
Esc
:w # File saved, vi stays open, keep editing

# Save a backup copy:
:w config_backup.txt # Saves current content to new file
```

■ **:q! is your emergency exit. If you accidentally edit a system file like /etc/hosts or /etc/passwd — press Esc then :q! immediately to abandon all changes.**

Navigation Commands (Normal mode)

h j k l

Move the cursor in Normal mode without arrow keys

What they do:

In Normal mode, hjkl move the cursor. Arrow keys also work in most vi versions but hjkl is the traditional vi way — your hands never leave the home row. In BusyBox vi or some minimal environments arrow keys may not work correctly.

Key	Direction	Arrow equivalent
h	Left	← Left arrow
j	Down	↓ Down arrow
k	Up	↑ Up arrow
l	Right	→ Right arrow

Additional navigation:

Command	What it does
0 (zero)	Jump to beginning of current line
\$	Jump to end of current line
gg	Jump to first line of file
G	Jump to last line of file
:N (number)	Jump to line number N — e.g. :25 jumps to line 25
w	Jump forward one word
b	Jump backward one word

Delete Commands (Normal mode)

dd dw d\$ x

Delete text in Normal mode

What they do:

Delete commands work in Normal mode — you do NOT need to be in Insert mode. Deleted content is placed in vi's buffer and can be pasted with p (below cursor) or P (above cursor).

All delete options:

Command	What it deletes
dd	Entire current line — the most used delete command
dw	From cursor to end of current word
d\$	From cursor to end of current line
d0	From cursor to beginning of current line
x	Single character under the cursor
X	Single character before the cursor (backspace equivalent)
Ndd	N lines from current line — e.g. 3dd deletes 3 lines

Examples on nezuko:

```
# Editing /home/scottp/config.txt:
dd # Delete the entire line cursor is on
3dd # Delete 3 lines at once
x # Delete single character under cursor (typo fix)
dw # Delete one word forward from cursor
d$ # Delete from cursor to end of line

# After deleting a line with dd:
p # Paste deleted line below cursor (move lines around!)
P # Paste deleted line above cursor
```

Undo Command (Normal mode)

u Ctrl+r

Undo and redo changes

What they do:

u undoes the last action in Normal mode. In vim, u can be pressed multiple times to undo multiple actions. In BusyBox vi, only a single undo is available — so be careful, especially when editing important files in containers.

Command	vim behaviour	BusyBox vi behaviour
u	Undo last action — press repeatedly for multiple undos	Single undo only — one press, that is all
Ctrl+r	Redo — reverses an undo	Not available
U	Undo all changes on current line	Not always available

■ In BusyBox vi (containers, embedded systems) you only get ONE undo. Think before you delete. Use :q! to abandon everything if you make a serious mistake.

Search Commands (Normal mode)

/ ? n N

Search for text within a file

What they do:

Search commands let you find text within the file without scrolling manually. Type the search term after / or ? and press Enter. Vi positions the cursor at the first match found.

Command	What it does
/searchterm	Search forward (downward) through the file
?searchterm	Search backward (upward) through the file
n	Jump to the next match in the same direction
N	Jump to the previous match (reverses direction)

Examples on nezuko:

```
sudo vi /etc/hosts
/nezuko # Search forward for 'nezuko'
/192.168 # Find IP address entries
n # Jump to next match
N # Jump back to previous match
?scottp # Search backward for 'scottp'

sudo vi /etc/netplan/00-installer-config.yaml
/dhcp # Find dhcp settings quickly
```

BusyBox — Minimal Vi in Containers

busybox vi

True minimal vi — what you find in Docker containers and embedded systems

What it is:

BusyBox is a single executable containing stripped-down versions of many Unix utilities including vi. It is used in Alpine-based Docker containers, embedded systems, routers, IoT devices, and rescue disks. This is the vi you will encounter in real production container environments.

```
sudo apt install busybox # Install BusyBox on nezuko
busybox vi test.txt # Open file with BusyBox vi
busybox --list # See all utilities BusyBox provides
```

What is missing compared to vim:

Missing Feature	Impact on you
No -- INSERT -- mode indicator	Must remember which mode you are in — Esc if unsure
No syntax highlighting	Plain text only — no colour coding
Single undo only	One u press — that is all you get
No Ctrl+r redo	Cannot redo after an undo
Limited colour support	Terminal looks very basic

■ All six essential commands (i, Esc, :wq, :q!, dd, u) work identically in BusyBox vi. Learn those six and you can edit files anywhere.

The Ubuntu vi Symlink — What You Are Really Running

On Ubuntu, vi is not a standalone program — it is a chain of symlinks pointing to vim. Follow the chain yourself on nezuko:

```
ls -la /usr/bin/vi
# Output: /usr/bin/vi -> /etc/alternatives/vi

ls -la /etc/alternatives/vi
# Output: /etc/alternatives/vi -> /usr/bin/vim.basic

# Chain: vi → /etc/alternatives/vi → /usr/bin/vim.basic
# You are running vim.basic every time you type vi on nezuko

vi --version | head -1 # Confirm which version you have
```

This is why vim features (multiple undo, syntax highlighting) work when you type vi on nezuko. In a Docker container or embedded system this symlink does not exist — you get true minimal vi instead.

Complete Workflow — Challenge Example on nezuko

Create a config file, make edits, delete a line, add a new line:

```
# Step 1 – Create and populate the file
vi config.txt # Open / create in /home/scottp/
i # Enter Insert mode
debug=false # Type first line then press Enter
max_connections=100 # Type second line
Esc # Return to Normal mode
:wq # Save and quit

# Step 2 – Re-open and edit
vi config.txt # Open the file again
dd # Delete first line (debug=false)
o # New line below, enter Insert mode
timeout=30 # Type new entry
Esc # Return to Normal mode
:wq # Save and quit

# Step 3 – Verify
cat config.txt
# Expected output:
# max_connections=100
# timeout=30
```

Quick Reference — All Essential Commands

Command	Mode	What it does
i	Normal → Insert	Insert before cursor
a	Normal → Insert	Append after cursor
o	Normal → Insert	New line below, enter Insert
O	Normal → Insert	New line above, enter Insert
A	Normal → Insert	Append to end of line
Esc	Insert → Normal	Return to Normal mode — always safe
:w	Normal	Save without quitting
:wq	Normal	Save and quit
:q	Normal	Quit (no unsaved changes only)
:q!	Normal	Quit and discard ALL changes
ZZ	Normal	Save and quit (shortcut)
h j k l	Normal	Navigate left / down / up / right
dd	Normal	Delete entire current line
dw	Normal	Delete word from cursor
x	Normal	Delete character under cursor
u	Normal	Undo (vim = multiple, BusyBox = one only)
Ctrl+r	Normal	Redo (vim only)
/term	Normal	Search forward for term
?term	Normal	Search backward for term
n / N	Normal	Next / previous search match
p / P	Normal	Paste below / above cursor
G / gg	Normal	Jump to last / first line of file
:N	Normal	Jump to line number N

■ Linux+ Exam Tips: Know the difference between Normal and Insert mode. Know :wq vs :q!. Know what BusyBox is and where you find it. Know vi exists everywhere but vim does not. Know hjkl navigation. Know that u has single undo only in BusyBox vi.